PROCESADORES DE LENGUAJES

# Práctica 3
# Creación de un compilador sencillo

Parte 1. Analizador Léxico

Curso 2024/2025

# 1. Introducción

Esta práctica será la última que realizaremos este curso, y por tanto la más extensa. Durante su realización aplicaremos todos los conocimientos adquiridos en la asignatura para la creación de un compilador para un lenguaje de programación sencillo cuya especificación se dará más adelante.

Nuestro compilador, tras la realización de las tres fases de análisis generará el AST del programa que se compile. Dejaremos fuera de esta práctica la generación de código final.

La implementación del compilador se podrá realizar en Python, java o C, a elección del grupo, aunque los ficheros que se proporcionen como guía estarán escritos en el lenguaje Python.

La práctica se dividirá en 3 partes, cada una de las cuales se centrarán en la implementación de cada uno de los analizadores. En esta primera parte nos centraremos en el analizador léxico.

# 2. Analizador Léxico

Empezaremos por implementar el analizador léxico de nuestro lenguaje. En esta implementación obviaremos algunas tareas del mismo que se añadirán más tarde, como la incorporación de los identificadores a la tabla de simbolos. Tras su implementación, y antes de continuar conviene verificar que nuestro analizador léxico funciona correctamente.

## 2.1. Especificación informal

En el aspecto léxico, nuestro lenguaje utilizará las categorías léxicas que utilizan la mayoria de los lenguajes, aunque en algunos casos su forma será diferente. A continuación haremos un repaso de las categorias léxicas que utilizaremos, haciendo una descripción informal de las mismas.

1. **Comentarios** Como todos los lenguajes el nuestro nos permitirá incluir comentarios que hagan más legibles nuestros programas. En nuestro lenguaje solamente tendremos comentarios de linea que empezarán por un doble símbolo de porcentaje ( % % ) y que terminarán al final de la linea donde se escribe.

2. Los espacios en blanco solamente se utilizan para separar los diferentes componentes léxicos. Se utilizarán tabulaciones y saltos de linea simplemente para hacer el código más legible.

3. Los identificadores **id** serán cadenas de letras y números que empiecen por una letra. Estos identificadores pueden tener una longitud máxima.

4. Los números que utilizaremos pueden ser tanto enteros como reales, y su definición viene dada por las siguiente expresiones regulares.

$$\begin{array}{ccc} digitos & \rightarrow & digito\ digito^* \\ fraccion_opt & \rightarrow & .\ digitos|\lambda \\ num & -> & digitos\ fraccion_opt \end{array}$$

5. El conjunto de palabras clave del lenguaje está formado por las siguientes palabras: PROGRAMA, VAR, ENTERO, REAL, BOOLEANO, INICIO, FIN, SI, ENTONCES, SINO, MIENTRAS, HACER, LEE, ESCRIBE, Y, O, NO, CIERTO y FALSO. Estas palabras reservadas deben estar escritas en mayúscula.

6. Los operadores que podremos utilizar en nuestro lenguaje son los siguientes.

   - Operadores relacionales son: =, <>, <, <=, >=, >

- Operadores aritméticos los representaremos como $+$, $-$, $*$, / y $\%$.
- El operador de asignacion es $:=$

7. Junto a las categorías indicadas anteriormente, en nuestro lenguaje utilizaremos también los siguientes símbolos: dos puntos (**:**), punto y coma (**;**), coma (**,**), y los parentesis de apertura y cierre ( **(** $y$ **)**)

## 2.2. Implementación del Analizador Léxico

Como ya comentamos en clase, una vez que tenemos definida la MDD que especifica las diferentes categorias léxicas, esta MDD debe ser implementada. Para ello, como ya comentamos en clase, tenemos dos opciones. La primera es definir la MDD mediante una tabla de adyacencia,y la segunda es implementar directamente en el lenguaje de programación el comportamiento de la MDD. Esta segunda opción será la que nosotros adoptaremos. Para conseguir una implementación completa de nuestro analizador léxico tendremos que abordar tres elementos que trataremos en los siguientes apartados.

### 2.2.1. Categorías Léxicas

Cada una de las categorías léxicas debemos implementarla como una clase. Cada clase tendrá unos atributos diferentes en función de la información que sobre esa categoría deberemos almacenar.

Todas estas clases heredarán de una clase genérica que representa una categoría general donde se agrupan los atributos y métodos comunes y que se da ya en el fichero componentes.py.

Debemos decidir que categorias léxicas vamos a utilizar. Para ello debemos decidir, por ejemplo, que vamos a hacer con los operadores relacionales. Podemos optar por definir una categoria para cada uno o definir una categoria en la que englobemos todos los operadores con un atributo que nos permita diferenciar un operador de otro.

Ejercicio 1

Completa el fichero componentes.py con la definición de todas las clases que representen las categorías léxicas que vayas a utilizar. Es conveniente que en cada componente se almacene el número de linea en el que se encuentra para posteriormente utilizar esa información en los mensajes de error.

### 2.2.2. El analizador

El analizador deberá leer la entrada carácter a carácter y agrupar estos caracteres en los diferentes elementos. Para ello, tras la lectura de un carácter, deberá comprobar en que rama de la MDD está para continuar por ella. Una vez que se llega el estado final de esa rama, el analizador debe devolver un objeto de la clase que se ha identificado. Este analizador lo implementaremos como una nueva clase que encapsulará el método que implementa el comportamiento de la MDD.

Un esqueleto de esta clase la podeis encontrar en el fichero analex.py que se os proporciona.

Ejercicio 2

Completa el fichero analex.py para que implemente el analizador léxico.

### 2.2.3. Tratamiento de errores léxicos

Durante la fase del análisis léxico solamente detectaremos errores de la aparición de caracteres no permitidos en el lenguaje.

## 3. Ejemplo

Para que podais ir probando vuestras implementaciones se proporciona el siguiente ejemplo

```
PROGRAMA p1;
VAR i,x:INTEGER;
INICIO
  i:=0.0;
   MIENTRAS (i<>5) HACER
    INICIO
      x:=i*4;
      ESCRIBE(x);
    FIN;
FIN.
```

Una vez completado vuestro analizador léxico la salida que debemos obtener sería la mostrada en 1.

```
PR (valor: PROGRAMA, linea: 1)
Identif (valor: p1, linea: 1)
PtoComa
PR (valor: VAR, linea: 2)
Identif (valor: i, linea: 2)
Coma
Identif (valor: x, linea: 2)
DosPtos
Identif (valor: INTEGER, linea: 2)
PtoComa
PR (valor: INICIO, linea: 3)
Identif (valor: i, linea: 4)
OpAsigna
Numero (valor: 0.0, linea: 4, tipo: <type 'float'>)
PtoComa
PR (valor: MIENTRAS, linea: 5)
ParentAp
Identif (valor: i, linea: 5)
OpRel (valor: SimDist, linea: 5)
Numero (valor: 5, linea: 5, tipo: <type 'int'>)
ParentCi
PR (valor: HACER, linea: 5)
PR (valor: INICIO, linea: 6)
Identif (valor: x, linea: 7)
OpAsigna
Identif (valor: i, linea: 7)
OpMult (valor: SimMult, linea: 7)
Numero (valor: 4, linea: 7, tipo: <type 'int'>)
PtoComa
PR (valor: ESCRIBE, linea: 8)
ParentAp
Identif (valor: x, linea: 8)
ParentCi
PtoComa
PR (valor: FIN, linea: 9)
PtoComa
PR (valor: FIN, linea: 10)
Punto
```

Figura 1: Salida del analizador léxico

## 4. Cuestiones de organización y evaluación

1. El tiempo estimado para el desarrollo del analizador léxico debe ser de 2 o 3 clases prácticas.
2. En la memoria final de la practica debe incluirse

- Especificación completa de todas las categorías léxicas del lenguaje y descripción del MDD equivalente. Deberá recoger las expresiones regulares que definen cada una de las categorías léxicas.
- **Detalles de implementación** Descripción de los aspectos más relevantes de la implementación del analizador léxico.

3. Una vez finalizada la implementación de esta parte, habrá una pequeña entrevista de no más de 10 minutos en la que se probará el analizador con diferentes ejemplos.